
Reliable Transfer on Lossy, High-Latency Links: Round-Based and Streaming NAK Retransmission over Paced UDP

Huacong Tang
University of Southern California

Xiao Yang
University of Southern California

Jiachen Lu
University of Southern California

 [Code](#)  [Demo Video](#)

Abstract

TCP prioritizes reliability and generality: its slow start, acknowledgment, and loss-as-congestion backoff are designed for unknown, shared networks. On dedicated point-to-point links, bandwidth, latency, and transfer size are known, and packet loss stems from the link rather than congestion. There, these mechanisms become the bottleneck: at a few percent random loss, TCP throughput collapses. We present a UDP-based reliable transport protocol for this setting. It removes congestion control and slow start and sends at a constant rate; loss never triggers slowdown. Reliability is instead guaranteed by a receiver-side bitmap and negative acknowledgments (NAKs) that retransmit only missing packets, with two control planes on the same data plane: a round-based one, where the sender pauses after each pass for a batched NAK, and a streaming one, where the sender never pauses and the receiver issues small periodic NAKs. On a testbed with 100 Mbps links, 200 ms RTT, and 20% injected loss, both deliver 1 GiB transfers at roughly $3000\times$ the throughput of TCP-based scp; the streaming control plane reaches 76 Mbps, within 3% of the erasure-channel bound, where the round-based one loses 13 s to feedback that its own shaper drops. Both require no tuning across loss rates and pass end-to-end integrity checks. These results suggest that on controlled links, abandoning probing mechanisms in favor of structural reliability guarantees is effective for high-loss transfer.

1 Problem and headline results

Move a 1 GiB file, one way, with zero errors, as fast as possible, over three impaired link configurations (Table 1). Reliability is a pass/fail gate proven by matching MD5 digests on both ends; speed is the competitive metric, with a 20 Mbps minimum (7.2 min for 1 GiB).

Figure 1 shows why the naive choices fail and where the opportunity is: over these same links, stock TCP (iperf) nearly vanishes—93 kbps under Case 2’s 20% random loss and only 15 Mbps under Case 3’s 200 ms RTT—while UDP delivers 78–97 Mbps regardless of loss or latency. TCP leaves the link almost empty; the UDP bars mark the ceiling our protocol targets.

Testbed. All experiments ran on three AWS EC2 instances (c7i-flex.large, Ubuntu 24.04) in one VPC — client 10.200.1.83, router, and server 10.200.2.48 on two subnets (10.200.1.0/24, 10.200.2.0/24), with every client–server packet forwarded through the router. Client and server

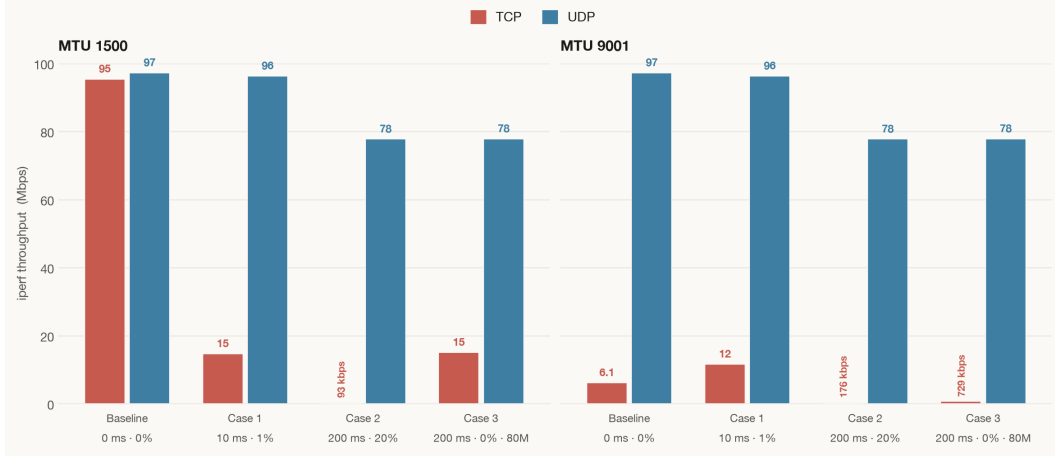


Figure 1: UDP ignores loss and RTT, so it barely changes across cases; TCP does not—Case 2 is loss-limited, Case 3 is window-limited by the high RTT.

Table 1: Headline results: 1 GiB one-way, MTU 1500, median of 3 repetitions per cell, MD5 verified on both ends in every run. Both control planes clear the 20 Mbps competition threshold by 3.4–4.8 $\times$; speedup over scp is 6.1 $\times$ (Case 1) and 3.1 $\times$ (Case 3), and Case 2 completes where scp cannot. The two control planes tie within 0.2 s in Cases 1 and 3; in Case 2 streaming is 12.5 s faster (§4). Loss is one-way data-path loss injected by netem at the router; timing is sender first-bit to receiver last-bit with measured clock-offset correction; min–max of the 3 runs is in Table 3.

Case	Link (rate / RTT / loss)	scp (TCP baseline)	Round-based NAK (§3.1)		Streaming NAK (§3.2)		Shannon $T_{\min}$
			time	goodput	time	goodput	
1	100 Mbps / 10 ms / 1%	548 s (15.7 Mbps)	90.0 s	95.4 Mbps	89.8 s	95.6 Mbps	88.9 s
2	100 Mbps / 200 ms / 20%	≈ 4 d (23 kbps)	124.8 s	68.8 Mbps	112.3 s	76.5 Mbps	110.0 s
3	80 Mbps / 200 ms / 0%	343 s (25.0 Mbps)	111.4 s	77.1 Mbps	111.3 s	77.2 Mbps	110.0 s

egress is rate-limited to 100 Mbps (tbf); the router applies each case’s rate limit plus netem delay/loss on both its interfaces, exactly as the lab specifies; the MTU is 1500 or 9001 on all interfaces per experiment.

2 Design by deletion: a fully known, dedicated link

This is not the open Internet. The link is point-to-point, uncontended, and every parameter (rate, RTT, loss probability) is known in advance; loss is injected at random by netem, so it does not signal congestion. Each known property removes a mechanism that classical transfer tools pay for (Table 2). TCP’s collapse in Table 1 and Figure 1 comes from treating random loss as congestion; we are free to ignore it.

Table 2: Design by deletion.

Known property	Mechanism removed
Dedicated link, no competing flows	Congestion control (AIMD / BBR)
Line rate known and fixed	Slow start, probing: pace at $\approx 0.99R$ from $t = 0$
File size known at start	Receiver preallocates; missing = bitmap complement
Loss random, not congestive	Backoff: loss never reduces the sending rate

3 Protocol

Both variants share one data plane and differ only in how the receiver’s feedback is scheduled. The data plane is UDP paced at the known line rate; the control plane is a missing-sequence list (NAK) from the receiver; TCP appears only at the edges of the session (HELLO, COMPLETE).

Data structures. Three in total. The sender holds an `mmap` of the source file, so blocks are sent directly from the page cache with no per-block copies. The receiver holds the preallocated output file and a bitmap of N bits (~ 90 KB for 1 GiB) marking which blocks have arrived. A NAK payload is a list of missing sequence numbers, i.e. part of the complement of the bitmap. The transfer ends only when the bitmap is full, so zero-error delivery is structural, not probabilistic, in both variants.

3.1 Round-based NAK: pause, collect, retransmit

Figure 2(a) shows one complete transfer, and Figure 3(a) gives the flow chart of the algorithm on each side. The sender paces all N blocks, then stops and asks for feedback; the receiver scans its bitmap and returns the whole missing list (3 copies over UDP); the sender retransmits exactly those blocks and asks again. Each round shrinks the missing set by the loss factor p , so the expected number of rounds is $\log_{1/p} N$ plus a small constant (Appendix A), and each round boundary costs the sender one RTT of idle time.

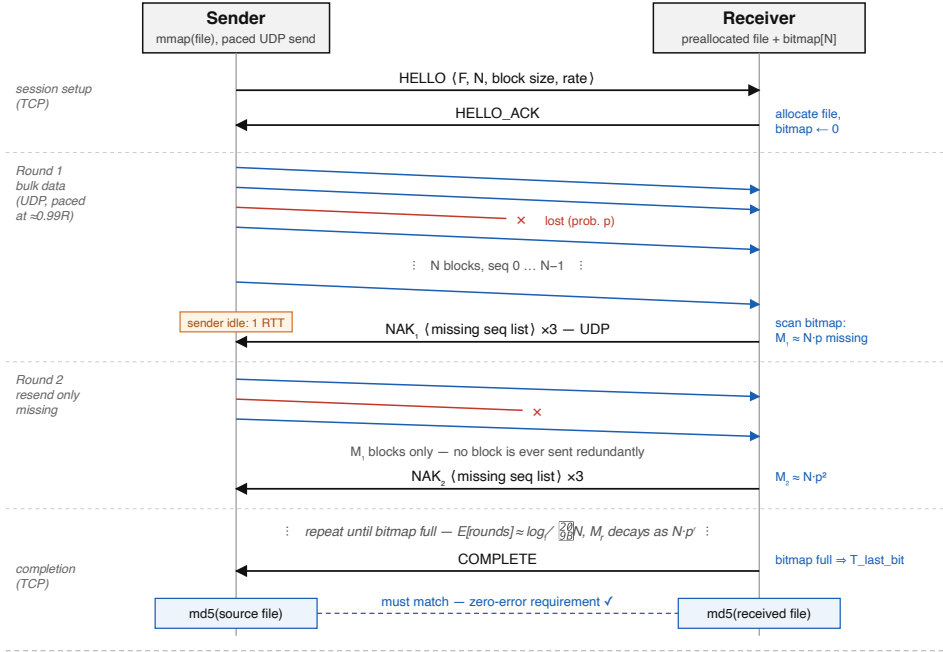
3.2 Streaming NAK: never pause

The round structure has one structural idle: at the end of every round the sender stops, and the next round cannot start until the NAK list has crossed the link and come back. The streaming variant removes that barrier and keeps everything else (Figures 2(b) and 3(b)).

Sender: one queue, no rounds. The sender drains a single queue at the paced rate. If a NAKed block is waiting it is retransmitted; otherwise the next fresh block is sent. It never stops until the first pass and every requested retransmission are done. Only then does it idle, sending a small “pass complete” marker every 10 ms while it waits for the last NAKs or the final COMPLETE. As before, loss never changes the sending rate. A block requested again within 5 ms of being sent is ignored: the two copies of one NAK arrive that close together, and the second copy is not new information.

Receiver: small, periodic, paced NAKs. Every 20 ms the receiver lists the blocks that should have arrived but have not: those below the highest first-pass sequence seen so far (minus a 32-block reordering guard), or every missing block once the pass-complete marker has arrived. A block is re-listed only after $\max(1.5 \times \text{RTT}, 50 \text{ ms})$ without arriving, so a retransmission already in flight is not requested twice; the RTT is a known link parameter and is passed in per case. Each NAK packet is sent twice with $150 \mu\text{s}$ spacing. Because repair is continuous, the outstanding list is a few hundred sequences at any moment. A NAK is therefore one or two frames rather than a burst, which matters for the reason given in §4.

(a) Round-based NAK: pause after each pass, batched NAK



(b) Streaming NAK: never pause, periodic small NAKs

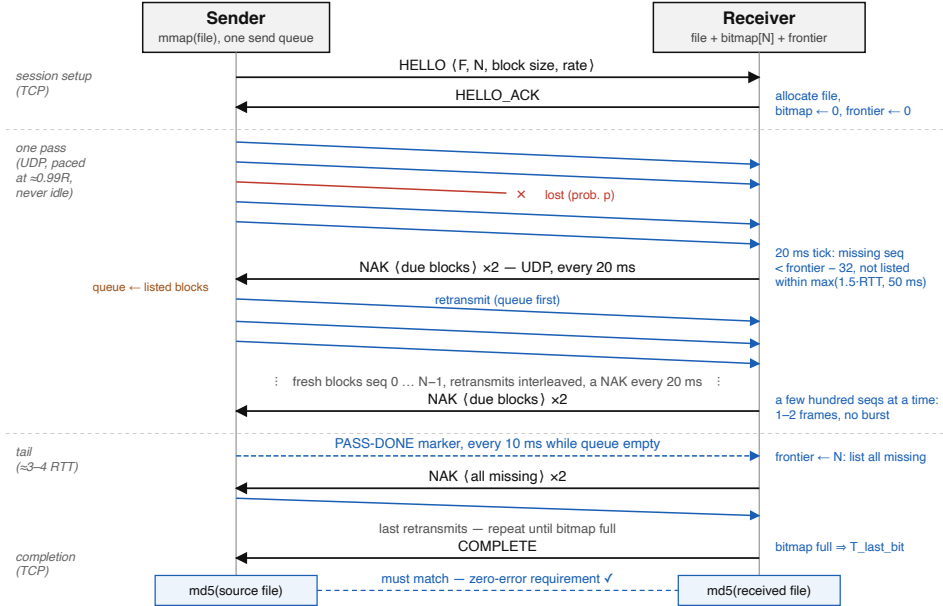


Figure 2: One transfer under each control plane. TCP carries only session setup and the final confirmation; data blocks (1464 B payload + 8 B header, filling a 1500 B frame; $N = 733,431$ blocks) flow over UDP, paced at the known rate; state is one bitmap (~ 90 KB) on the receiver and an mmap of the file on the sender. (a) Round-based: after each pass the sender idles for one RTT while the receiver returns its whole missing list (NAK, 3 copies); the sender retransmits exactly those blocks. (b) Streaming: the receiver sends a small NAK every 20 ms for the blocks that are due, and the sender interleaves those retransmissions into the pass; the only idle time is the tail after the pass.

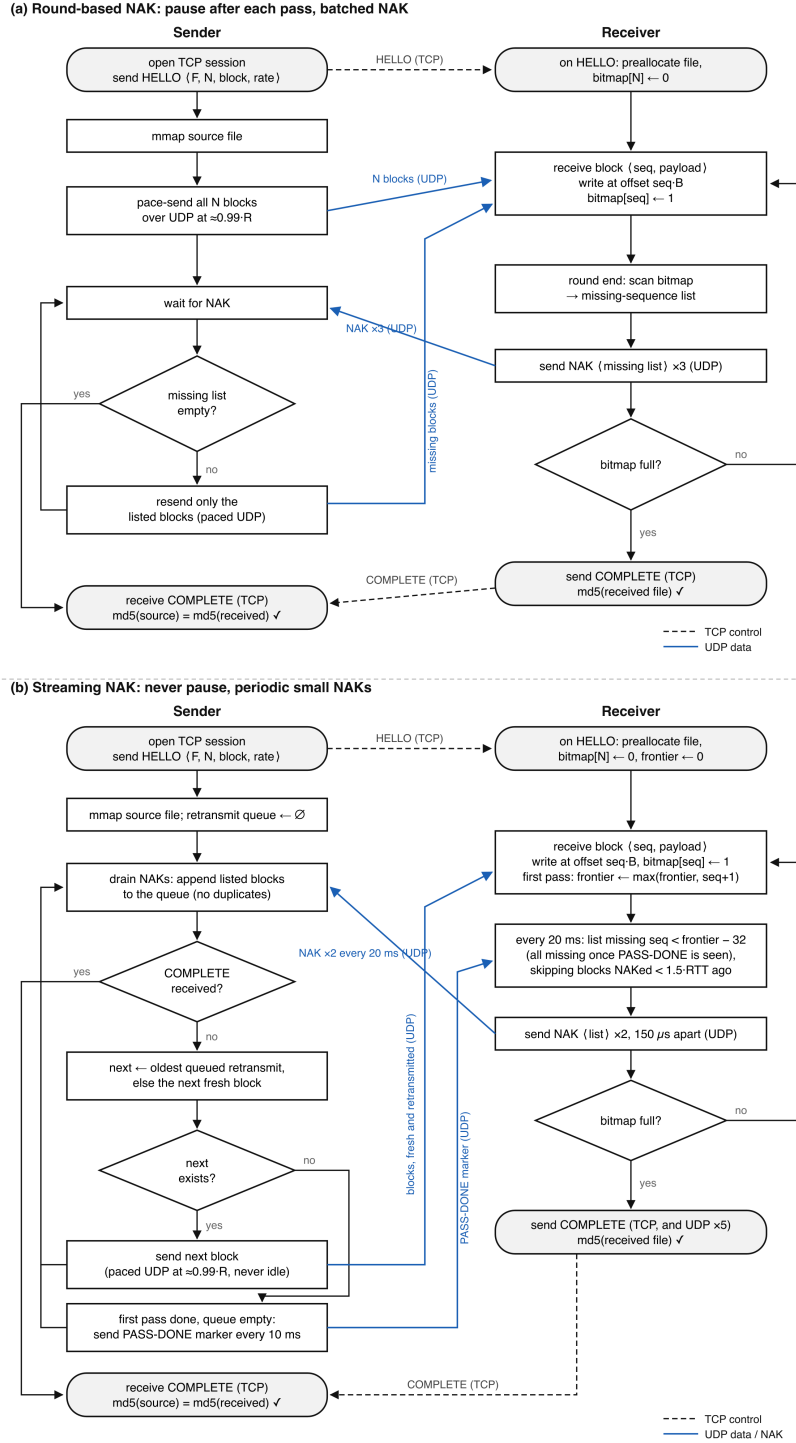


Figure 3: Flow charts of the two control planes. Solid rectangles: processing; diamonds: decisions; dashed black: TCP control; blue: UDP data and NAK path. (a) Round-based: the sender has a “wait for NAK” state and retransmits only the blocks named in the latest NAK. (b) Streaming: the sender drains NAKs into a queue and always has something to send until the pass and the queue are both exhausted; the receiver’s NAK is a 20 ms timer, not a round boundary. Both receiver loops end only when the bitmap is full.

4 Measurements

Figure 4 summarizes the measured one-way times for scp and both control planes; Figure 5 shows how each transfer completes over time; Figure 6 shows the same runs as goodput against the measured UDP delivery ceiling (Figure 1). Table 3 gives the per-case detail for both control planes.

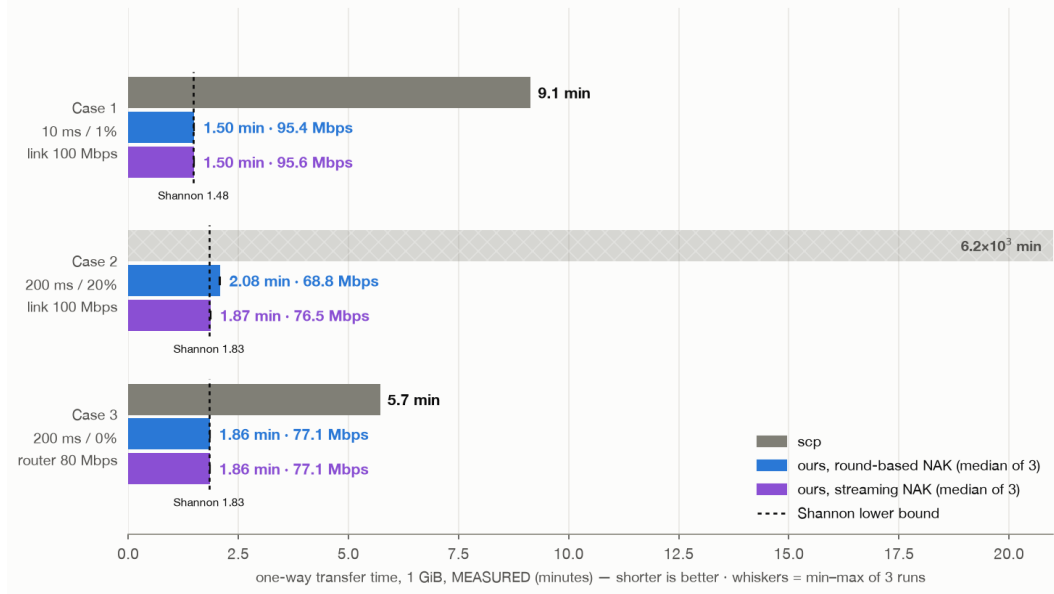


Figure 4: One-way transfer time for 1 GiB (3 reps/case). scp: 548 s / ≈ 4 d / 343 s; round-based: 90–125 s; streaming: 90–112 s. Only Case 2 separates the two control planes.

Table 3: Both control planes, MTU 1500, tuned pacing (99 Mbps; 79.5 Mbps in Case 3), three runs each, every run MD5-verified. Packets sent is the mean over the three runs; the ideal is $N/(1-p)$ (733,431 / 916,789 / 733,431). Feedback is NAK messages received by the sender (round-based: one per round; streaming: one 20 ms tick, two packets).

Case	Control plane	Time, med. [min–max]	Goodput	Packets sent	Rounds / NAK ticks	Gap to $T_{\min}$
1	round-based	90.0 s [90.0–90.0]	95.4 Mbps	740,880 (+1.0%)	5	+1.2%
	streaming	89.8 s [89.8–89.9]	95.6 Mbps	740,908 (+1.0%)	3,650	+1.1%
2	round-based	124.8 s [124.7–125.3]	68.8 Mbps	917,325 (+0.06%)	45	+13%
	streaming	112.3 s [112.2–112.5]	76.5 Mbps	916,754 (−0.0%)	5,585	+2.1%
3	round-based	111.4 s [111.3–111.4]	77.1 Mbps	735,565 (+0.3%)	2	+1.2%
	streaming	111.3 s [111.3–111.5]	77.2 Mbps	735,648 (+0.3%)	1,798	+1.2%

Where the round-based variant loses 13 s in Case 2. In Cases 1 and 3 both control planes land within 1.2% of the erasure bound and within 0.2 s of each other: with 1% or 0% loss there are only a few rounds, so the round barrier costs almost nothing. Case 2 is different. The round-based sender logs show 45 rounds where the model of Appendix A predicts 9. The cause is not the 20% loss on the return path but the shape of the first NAK. After round 1 about 147k blocks are missing; the list is 406 UDP segments, sent three times back to back, and this burst of ≈ 1200 packets hits the receiver’s own egress shaper (tbf with a one-frame burst allowance). In the logged run only 20 of the 406 segments survived, so round 2 could repair only 6,944 blocks instead of 147k. The transfer degenerated into 37 rounds of 4–7k blocks, each paying one RTT plus the sender’s feedback window of idle time. That idle time is about 13 s in total, which is the entire gap between the measured 124.8 s and the 111.1 s the frames occupy on the wire. The streaming variant never builds such a list: its NAKs are one or two frames every 20 ms, and its sender is never idle before the first pass ends. It finishes the first pass at 111.0–111.1 s and the tail of remaining repairs takes 1.3–1.6 s, so the transfer sits 1.1% above the wire time and 2.1% above the 110.0 s Shannon bound. Duplicate retransmissions (blocks whose repair was still in flight when the re-list guard expired) are below the run-to-run noise: the three runs sent 916,754 frames on average against the 916,789 expected for 20% loss.

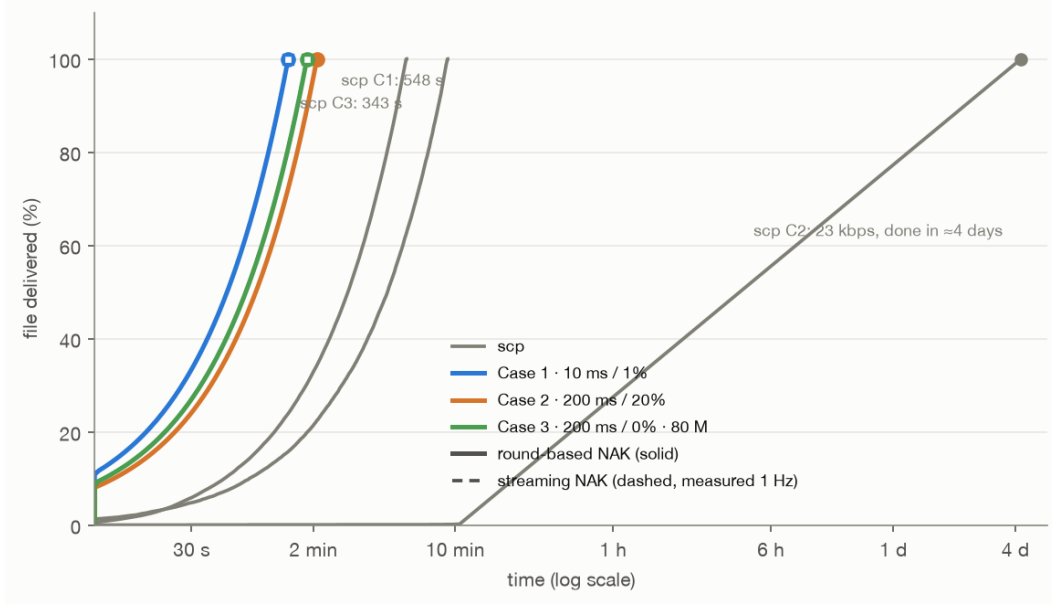


Figure 5: Cumulative delivery on a log time axis. Color: our protocol, one representative run per case; solid: round-based (curve reconstructed from per-round logs, scaled to the measured duration); dashed: streaming (receiver’s measured 1 Hz trace); gray: scp per-second progress. In linear time the colored curves are straight lines: the delivery rate is constant end to end, because pacing never slows on loss and every retransmitted packet is a needed one. scp Case 2 crawls at 23 kbps and completes only after ≈ 4 days (§4).

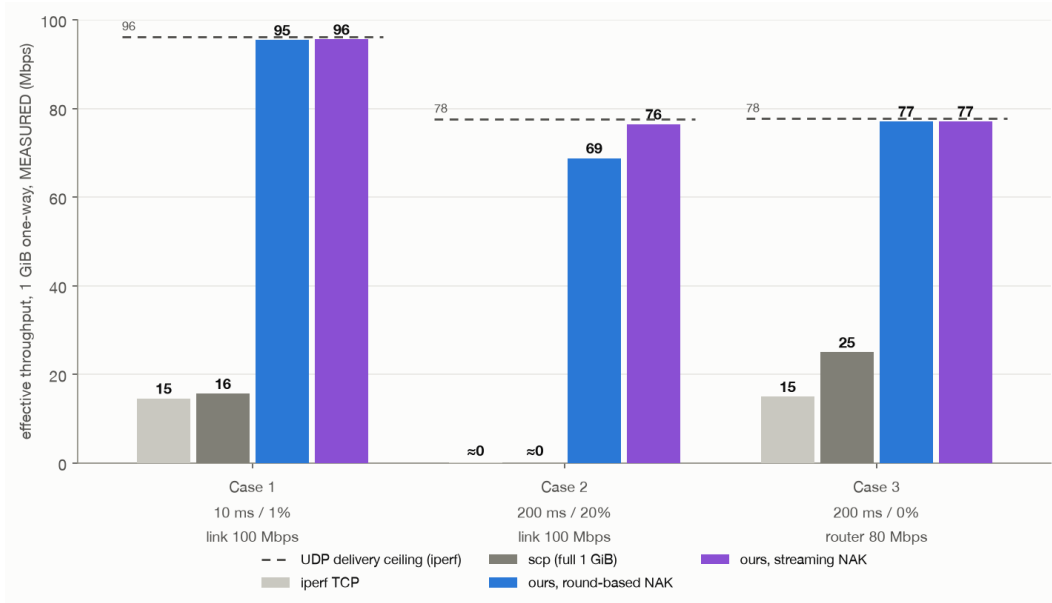


Figure 6: Goodput vs. the measured UDP delivery ceiling (dashed): both control planes operate at the ceiling in Cases 1 and 3; in Case 2 the round-based variant sits below it and the streaming variant closes the gap. The remaining gap is retransmission ($= p_{\text{eff}}$) plus header overhead. iperf TCP and scp shown for reference.

Why scp cannot finish (Case 2). scp’s steady state on this link is 23 kbps, i.e. ≈ 4 days for 1 GiB: $\sim 900\times$ below the competition threshold and $\sim 3000\times$ slower than our protocol. Kernel telemetry (`ss -tin`) shows the mechanism: `cwnd` stays pinned at 1–2 (`ssthresh` collapses to 7), so three duplicate ACKs never accumulate, fast retransmit never triggers, and every loss is repaired by timeout; the RTO backs off exponentially and each repair costs a full timeout. The sender averages 1.9 packets/s, 0.02% of the link’s ~ 8300 packet/s capacity, with $\approx 23\%$ of sent bytes being retransmissions, matching the 20% loss rate. TCP here is not bandwidth-limited: interpreting random loss as congestion, it throttles itself to idleness.

Robustness. A pacing sweep at 85–99 Mbps (Case 2, round-based) falls monotonically: sending faster adds `tbh-overflow` retransmissions, but the higher rate always wins, so the headline runs of both variants pace at 99 Mbps (0.99*R*; 79.5 Mbps in Case 3 under its 80 Mbps bottleneck). The testbed adds almost nothing beyond the configured impairment — p_{eff} measured from packet counts is 20.0–20.5% against Case 2’s configured 20%, and within 0.7% of configuration in the other cases — and reliability was never affected: every transfer of either variant converged and passed MD5.¹

5 MTU 9001 vs. 1500

Jumbo frames run natively on the same topology, and they are *faster* in every case for both control planes (Figure 7): the header overhead falls ($\eta = 1464/1500 = 0.976 \rightarrow 8965/9001 = 0.996$) and the same rate needs $6\times$ fewer packets per second. Measured p_{eff} stays at the configured loss for both MTUs, so the specified burst 9015 shaper passes paced jumbo UDP cleanly. The fastest round-based configuration, Case 1 at MTU 9001, reaches 96.8 Mbps, within 1.5% of that link’s erasure bound; the streaming variant reaches 96.9 Mbps there. In Case 2 at MTU 9001 the round-based variant already recovers most of its Case 2 deficit (75.8 Mbps): with $6\times$ fewer blocks the round-1 NAK is $6\times$ shorter and mostly survives the shaper. The streaming variant reaches 77.8 Mbps at MTU 9001 in Case 2, 1.1% above that link’s bound. Case 3 at MTU 9001 is the one cell where streaming is slower (76.8 vs. 77.8 Mbps): with 0% configured loss its 2.6% retransmissions are all shaper drops, against 0.7% for the round-based sender. Our reading of the counts is pacing jitter: the streaming sender polls for NAKs between packets, and at 9001-byte frames the one-frame burst 9015 bucket has no slack to absorb the extra delay, so it drops a jumbo frame now and then; at MTU 1500 the same jitter is absorbed. This is a pacer tuning issue rather than a protocol one, and we did not tune it.

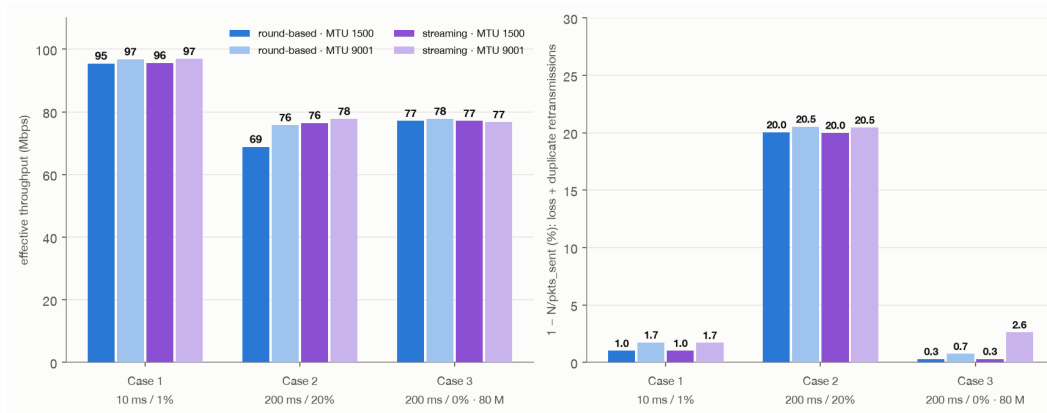


Figure 7: Left: goodput with tuned pacing, median of 3 runs per bar, all MD5-verified — jumbo is faster in every case for both control planes. Right: $1 - N/\text{packets sent}$, i.e. the fraction of frames that were not first deliveries. For the round-based variant this is the effective loss p_{eff} and matches the configured loss; for the streaming variant it also includes its duplicate retransmissions.

¹1 GiB test file MD5 2322915a33d9a0fc4f1dab8a8b795c8f for the round-based runs; the streaming runs, made on a later instance of the same topology, used a regenerated 1 GiB file (MD5 03afa2ce8cb80ef6475790ff34716cb6). Both digests were verified sender- and receiver-side in every reported run.

6 Conclusion

On a dedicated link with known parameters, reliability is cheap: pace at line rate, and retransmit only what the bitmap says is missing. Both control planes deliver 1 GiB in about two minutes in every case: 3.1–6.1 $\times$ faster than scp where scp finishes at all. Where they differ is instructive. The round-based variant is within 1.2% of the erasure-channel limit in the two low-loss cases but 13% above it in Case 2, and the reason is not the loss rate itself but a burst of feedback that the receiver’s own shaper drops. The streaming variant, which never pauses and never bursts, is within 1.1–2.1% of the limit in all three cases. With that, the overhead beyond the Shannon bound is 1–3 s in every case, so no FEC or further protocol machinery can improve it meaningfully.

A Analysis: provably near-optimal

We work through Case 2 ($R = 100$ Mbps, $\text{RTT} = 200$ ms, $p = 0.20$); Cases 1 and 3 follow from the same expressions.

Setup. The file is $F = 1$ GiB $= 2^{30}$ B $= 8,589,934,592$ bits. Each block carries 1464 B of payload in a 1500 B frame, so the payload efficiency is $\eta = 1464/1500 = 0.976$ and the block count is

$$N = \lceil 2^{30}/1464 \rceil = 733,431. \quad (1)$$

Lower bound. A link that drops each frame independently with probability p is an erasure channel of capacity $(1 - p)R$, of which a fraction η is payload. No protocol or code can deliver the file faster than

$$\begin{aligned} T_{\min} &= \frac{F}{(1 - p)\eta R} \\ &= \frac{8,589,934,592 \text{ b}}{0.80 \times 0.976 \times 10^8 \text{ b/s}} = \frac{8,589,934,592}{7.808 \times 10^7} \text{ s} = 110.0 \text{ s}. \end{aligned} \quad (2)$$

The protocol meets the bound in expectation. Each block must be received once. A given transmission survives with probability $1 - p$, and the bitmap guarantees no block is re-sent after it has been received, so the number of transmissions of one block is geometric with mean $1/(1 - p)$ and the total is

$$\mathbb{E}[\text{packets}] = N(1 + p + p^2 + \dots) = \frac{N}{1 - p} = \frac{733,431}{0.80} \approx 916,789. \quad (3)$$

Paced back-to-back at the line rate, these frames occupy the wire for

$$\frac{916,789 \times 1500 \times 8 \text{ b}}{10^8 \text{ b/s}} = 110.0 \text{ s} = T_{\min}, \quad (4)$$

i.e. constant-rate sending plus retransmit-only-what-is-missing wastes no transmission in expectation. This holds for both control planes; the only cost left is how the feedback is scheduled.

Round overhead. Round 1 sends all N blocks; round $k+1$ re-sends the expected Np^k blocks still missing. For Case 2 the expected round sizes, each 0.2 $\times$ the previous, are

$$733,431 \rightarrow 146,686 \rightarrow 29,337 \rightarrow 5,867 \rightarrow 1,173 \rightarrow 235 \rightarrow 47 \rightarrow 9 \rightarrow 2 \rightarrow <1. \quad (5)$$

The transfer ends when the slowest block has arrived, i.e. after the maximum of N i.i.d. geometric variables, whose expectation is

$$\begin{aligned} \mathbb{E}[\text{rounds}] &\approx \log_{1/p} N + \frac{\gamma}{\ln(1/p)} + \frac{1}{2} \\ &= \frac{\ln 733,431}{\ln 5} + \frac{0.5772}{\ln 5} + 0.5 = 8.39 + 0.36 + 0.50 = 9.3 \end{aligned} \quad (6)$$

(γ is the Euler–Mascheroni constant). Each round boundary costs one RTT of sender idle time (the last block’s propagation plus the NAK’s return), so the expected overhead beyond $T_{\min}$ is

$$\mathbb{E}[\text{rounds}] \cdot \text{RTT} = 9.3 \times 0.2 \text{ s} = 1.9 \text{ s}, \quad 1.7\% \text{ of } T_{\min}. \quad (7)$$

Streaming overhead. The streaming sender is idle only after the first pass, while the last few missing blocks make their final round trips. The last block needs, in expectation, the same $\log_{1/p}$ number of attempts, but only the attempts after the pass ends cost idle time: with ≈ 1700 frames per RTT in flight and 20% loss, a few hundred blocks are outstanding when the pass ends, and clearing them takes about $\log_{1/p}(300) \approx 3.5$ round trips, i.e. ≈ 0.7 s plus the NAK tick and re-list guard. The expected overhead is therefore ≈ 1 s, about half of the round-based figure, and it does not grow with N .

Predicted times. Hence

$$T \approx T_{\min} + \mathbb{E}[\text{rounds}] \cdot \text{RTT} = 110.0 + 1.9 = 111.9 \text{ s} \quad (8)$$

for Case 2 round-based; the same computation gives 88.9 s for Case 1 (+0.04%) and 110.2 s for Case 3 (+0.2%). That ~ 2 s is also the ceiling on what any FEC scheme could save, which is why we use none. Cases 1 and 3 land within 1.2% of this prediction (measured 90.0 s and 111.4 s for round-based; 89.8 s and 111.3 s for streaming). Case 2's round-based 124.8 s exceeds it because the idealized model assumes every NAK is delivered. The round-1 NAK is a burst of ≈ 1200 packets and the receiver's own tbf shaper drops most of it (20 of 406 segments delivered in the logged run), so the transfer takes 45 rounds rather than 9.3, and each extra round costs an RTT plus the sender's feedback window (≈ 35 rounds $\times$ 0.3–0.4 s ≈ 13 s; §4). Measured p_{eff} equals the configured 20% (Figure 7), confirming the excess is round overhead, not extra loss. The streaming variant, which has no round boundary, measures 112.3 s, within 0.4 s of the 111.9 s prediction; the residual is the 99 Mbps pacing rate ($110.0/0.99 = 111.1$ s of wire time) plus a 1.3–1.6 s tail.